

缓存测量实验

测量逻辑

1. 测量缓存大小

负载程序 `sample_cache_size.cpp` 中涉及到对一个大数组的多次访问：进行多次循环，每轮循环从下标0访问至下标为传入的参数`testSize`的项（其中会根据参数`testSize`调整步长，保证不同的参数，访问数组的次数相同）。在 `MeasureCacheSize` 中采用可能的所有参数（从小到大）多次运行 `sample_cache_size.cpp` 程序，并记录每次运行的时间。总访问次数相同的情况下，如果访问的最大的下标从小于`cache`变为大于`cache`，则会造成读缺失（需要进行替换），此时读取所花平均时间增大，程序的整体运行时间也大幅增大，在 `MeasureCacheSize` 中计算下一次测试的`cache size`的运行时间与当前测试的运行时间的差，则差值最大发生跃升对应的`testSize`，即为模拟器的数据缓存大小。

2. 测量缓存行大小

负载程序 `sample_block_size.cpp` 与前面类似，涉及到对大数组的访问（但是只需要进行一次循环）。程序可以接收一个传入参数表示步长，程序保持整体的数组访问次数一定，根据不同的步长进行数组的访问。注意到最大的`cache size`为4096，最小的`block size`为4，因此设置访问次数为4096总可以保证在任何情况下都能产生L1 Dcache的替换。在 `MeasureCacheBlockSize` 同样由小到大遍历所有可能的`Block size`，多次运行负载程序并记录运行时间。随着测试的`step`增大，替换次数逐渐增大，整体花费的时间会增大。注意模拟器的内存访问的机制使得连续的内存访问 Memory会立即返回结果，因此当`step`小于等于`block size`时，替换时花费的时间不会很多，而如果`step`变到大于`block size`，此时若进行替换，内存访问不再是连续的，因此替换所花的访问时间将大幅增大，造成整体周期数陡增。因此类似上面同样计算运行时间差，发生陡增时的前一项对应的测试`step`即为`block size`。

3. 测试cache的相联度

负载程序 `sample_associativity` 接收`cacheSize`, `cacheBlockSize`, `testAssociativity`为参数，并进行数组的循环访问。考虑到可能的相联度只为1, 2, 4, 8, 16，可以以步长为 $(\text{cacheLineCount}/\text{testAssociativity}) * \text{cacheBlockSize}$ 访问`char`型数组（从0访问至两倍`cacheSize`大小，需保证总的访问次数相同），当测试的相联度较小时，步长较大，所访问的元素总会落在第一组`cache`中，但是不会填满第一组`cache`（因为步长较大，访问的不同元素的个数不会比相联度大），而当测试的关联度等于实际相联度时，步长即为一个`cache`组的数目，由于访问到2倍`cacheSize`，则不同元素的个数会是相联度的两倍，此时会发生大量的替换，整体时间陡增。因此在 `MeasureCacheAssociativity` 中同样可以确定，周期数陡增时的`testAssociativity`即为实际的相联度（另外考虑相联度为1的特殊情况，实际上每一次`test`都会有大量的替换，所有的运行时间相差都不大，因此可以先令`index=0`，并设置一个阈值，只有大于这个阈值的增长才被考虑）

4. 优化矩阵乘法

原本的矩阵乘法最内层循环计算的是 `c[i][j]`，然而由于矩阵乘法的计算法则，需要访问第二个矩阵的第`j`列，而二维数组采用的存储是行优先，对矩阵的列的访问不是连续的，此时容易出现许多`cachemiss`。因此考虑改变矩阵访问方式——尽量按行访问。考虑矩阵乘法的理解，可以理解为左阵的第`i`行元素，对右阵的每一行进行加权，进而得到结果矩阵的第`i`行。因此可以仍然保持外两层循环为`i`以及`j`，但是最内层的循环是使用 `a[i][j]` 乘以`b`矩阵的第`j`行，并将得到的结果分别加到结果矩阵的第`i`行的每一个元素，这样最内层循环之访问量`a`的一个元素，而对`b`以及`c`矩阵的元素访问都是按行进行（访问一整行），避免了列访问，减少了`cachemiss`，在实际结果上也提升了速度。